SiyaBusa ERP

API DOCUMENTATION

Technical Documentation

Masaphokati Technologies (Pty) Ltd

Document Version: 1.0

Effective Date: January 2026

Department: Engineering

Classification: Confidential

WE RULE. WE GOVERN. WE EMPOWER.

SiyaBusa ERP

> **DEVELOPER REFERENCE**
>
> Complete REST API documentation with authentication, endpoints, and examples.

API Documentation

Table of Contents

Introduction

Overview

The SiyaBusa ERP API provides programmatic access to your business data and operations. Built on RESTful principles, our API enables you to:

- Integrate with third-party applications

- Build custom dashboards and reports
- Automate business workflows
- Sync data with external systems
- Create mobile and web applications

API Design Principles

Principle	Description
RESTful	Resource-based URLs with standard HTTP methods
JSON	All requests and responses use JSON format
Multi-tenant	Tenant context required for all operations
Versioned	API versioning via URL path (v2)
Secure	OAuth 2.0 authentication with JWT tokens

Getting Started

1. Obtain API credentials from your SiyaBusa admin portal
2. Authenticate to receive access token
3. Include token in Authorization header
4. Make API calls to desired endpoints

Authentication

OAuth 2.0 Flow

SiyaBusa API uses OAuth 2.0 with JWT tokens for authentication.

Client Credentials Flow (Server-to-Server)

```
POST /api/v2/auth/token
Content-Type: application/json

{
  "grant_type": "client_credentials",
  "client_id": "your_client_id",
  "client_secret": "your_client_secret",
  "tenant_id": "your_tenant_id"
}
```

Response

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "dGhpcyBpcyBhIHJlZnJlc2ggdG9rZW4...",
  "scope": "read write"
}
```

Using the Access Token

Include the token in the Authorization header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Token Refresh

```
POST /api/v2/auth/refresh
Content-Type: application/json

{
  "grant_type": "refresh_token",
  "refresh_token": "dGhpcyBpcyBhIHJlZnJlc2ggdG9rZW4..."
}
```

Scopes

Scope	Description
`read`	Read access to resources
`write`	Create and update resources
`delete`	Delete resources
`admin`	Administrative operations
`financial:read`	Read financial data
`financial:write`	Modify financial data
`hr:read`	Read HR data
`hr:write`	Modify HR data

Base URL & Environments

Environments

Environment	Base URL	Purpose
Production	`https://api.siyabusa.com/api/v2`	Live data
Sandbox	`https://sandbox.siyabusa.com/api/v2`	Testing
Staging	`https://staging.siyabusa.com/api/v2`	Pre-production

API Versioning

The API version is included in the URL path:

```
https://api.siyabusa.com/api/v2/customers
                          ^^
                          Version
```

Current version: **v2**

Request & Response Format

Request Headers

Header	Required	Description
`Authorization`	Yes	Bearer token
`Content-Type`	Yes	`application/json`
`X-Tenant-ID`	Yes	Your tenant identifier
`X-Request-ID`	No	Unique request ID for tracing
`Accept-Language`	No	Response language (default: en)

Standard Request Format

```
GET /api/v2/customers?page=1&limit=20
Host: api.siyabusa.com
Authorization: Bearer {token}
Content-Type: application/json
X-Tenant-ID: tenant_abc123
```

Standard Response Format

```
{
  "success": true,
  "data": {
    // Response data
  },
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 150,
    "totalPages": 8
  },
  "timestamp": "2026-01-03T10:30:00Z",
  "requestId": "req_abc123xyz"
}
```

Pagination

All list endpoints support pagination:

Parameter	Default	Max	Description
`page`	1	-	Page number
`limit`	20	100	Items per page
`sort`	`created_at`	-	Sort field
`order`	`desc`	-	Sort order (asc/desc)

Filtering

Use query parameters for filtering:

```
GET /api/v2/invoices?status=paid&date_from=2026-01-01&date_to=2026-01-31
```

Including Related Resources

Use the `include` parameter:

```
GET /api/v2/invoices/123?include=customer,line_items,payments
```

Error Handling

Error Response Format

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The request contains invalid data",
    "details": [
      {
        "field": "email",
        "message": "Email is required"
      },
      {
        "field": "amount",
        "message": "Amount must be greater than 0"
      }
    ]
  },
  "timestamp": "2026-01-03T10:30:00Z",
  "requestId": "req_abc123xyz"
}
```

HTTP Status Codes

Code	Meaning	Description
200	OK	Request successful
201	Created	Resource created
204	No Content	Successful, no response body
400	Bad Request	Invalid request data
401	Unauthorized	Authentication required
403	Forbidden	Insufficient permissions
404	Not Found	Resource not found
409	Conflict	Resource conflict
422	Unprocessable Entity	Validation error
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Server error

Error Codes

Code	Description
`AUTH_REQUIRED`	Authentication token missing
`AUTH_INVALID`	Invalid or expired token
`AUTH_FORBIDDEN`	Insufficient permissions
`TENANT_REQUIRED`	Tenant ID missing
`TENANT_INVALID`	Invalid tenant ID
`VALIDATION_ERROR`	Request validation failed
`RESOURCE_NOT_FOUND`	Requested resource not found
`RESOURCE_EXISTS`	Resource already exists
`RATE_LIMIT_EXCEEDED`	Too many requests
`INTERNAL_ERROR`	Server error

Rate Limiting

Limits

Plan	Requests/Minute	Requests/Day
Starter	60	10,000
Professional	300	50,000
Enterprise	1,000	200,000

Rate Limit Headers

```
X-RateLimit-Limit: 300
X-RateLimit-Remaining: 287
X-RateLimit-Reset: 1704282600
```

Rate Limit Exceeded Response

```
{
  "success": false,
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Rate limit exceeded. Please retry after 45 seconds.",
    "retryAfter": 45
  }
}
```

Core API Endpoints

Tenant Information

```
GET /api/v2/tenant
```

Returns current tenant information.

Response:

```
{
  "success": true,
  "data": {
    "id": "tenant_abc123",
    "name": "ABC Company (Pty) Ltd",
    "plan": "professional",
    "createdAt": "2024-03-15T00:00:00Z",
    "settings": {
      "currency": "ZAR",
      "timezone": "Africa/Johannesburg",
      "dateFormat": "DD/MM/YYYY",
      "financialYearStart": "03-01"
    }
  }
}
```

Users

Method	Endpoint	Description
GET	`/users`	List all users
GET	`/users/{id}`	Get user by ID
POST	`/users`	Create user
PUT	`/users/{id}`	Update user
DELETE	`/users/{id}`	Deactivate user

Audit Logs

```
GET /api/v2/audit-logs?resource=invoice&action=create&date_from=2026-01-01
```

Financial Module API

Chart of Accounts

Method	Endpoint	Description
GET	`/financial/accounts`	List all accounts
GET	`/financial/accounts/{id}`	Get account
POST	`/financial/accounts`	Create account
PUT	`/financial/accounts/{id}`	Update account
GET	`/financial/accounts/{id}/transactions`	Account transactions

Create Account:

```
POST /api/v2/financial/accounts

{
  "code": "1000",
  "name": "Cash on Hand",
  "type": "asset",
  "subType": "current_asset",
  "currency": "ZAR",
  "isActive": true,
  "parentAccountId": null
}
```

General Ledger

Method	Endpoint	Description
GET	`/financial/journal-entries`	List journal entries
GET	`/financial/journal-entries/{id}`	Get journal entry
POST	`/financial/journal-entries`	Create journal entry
POST	`/financial/journal-entries/{id}/post`	Post entry
POST	`/financial/journal-entries/{id}/reverse`	Reverse entry

Create Journal Entry:

```
POST /api/v2/financial/journal-entries

{
  "date": "2026-01-03",
  "reference": "JE-001",
  "description": "Monthly rent payment",
  "lines": [
    {
      "accountId": "acc_123",
      "debit": 15000.00,
      "credit": 0,
      "description": "Rent expense"
    },
    {
      "accountId": "acc_456",
      "debit": 0,
      "credit": 15000.00,
      "description": "Bank payment"
    }
  ]
}
```

Accounts Payable

Method	Endpoint	Description
GET	`/financial/bills`	List bills
POST	`/financial/bills`	Create bill
POST	`/financial/bills/{id}/payments`	Record payment
GET	`/financial/suppliers`	List suppliers
GET	`/financial/ap-aging`	AP aging report

Accounts Receivable

Method	Endpoint	Description
GET	`/financial/invoices`	List invoices
POST	`/financial/invoices`	Create invoice
POST	`/financial/invoices/{id}/payments`	Record payment
GET	`/financial/ar-aging`	AR aging report

Bank Reconciliation

Method	Endpoint	Description
GET	`/financial/bank-accounts`	List bank accounts
POST	`/financial/bank-accounts/{id}/transactions`	Import transactions
POST	`/financial/bank-accounts/{id}/reconcile`	Reconcile transactions
GET	`/financial/bank-accounts/{id}/statement`	Bank statement

Financial Reports

Method	Endpoint	Description
GET	`/financial/reports/trial-balance`	Trial balance
GET	`/financial/reports/income-statement`	Income statement
GET	`/financial/reports/balance-sheet`	Balance sheet
GET	`/financial/reports/cash-flow`	Cash flow statement

Report Parameters:

```
GET /api/v2/financial/reports/income-statement?
    from_date=2026-01-01&
    to_date=2026-01-31&
    compare=previous_period&
    format=detailed
```

Inventory Module API

Products

Method	Endpoint	Description
GET	`/inventory/products`	List products
GET	`/inventory/products/{id}`	Get product
POST	`/inventory/products`	Create product
PUT	`/inventory/products/{id}`	Update product
GET	`/inventory/products/{id}/stock`	Stock levels
GET	`/inventory/products/{id}/movements`	Stock movements

Create Product:

```
POST /api/v2/inventory/products

{
  "sku": "WIDGET-001",
  "name": "Standard Widget",
  "description": "High-quality standard widget",
  "category": "Widgets",
  "unitOfMeasure": "each",
  "costPrice": 50.00,
  "sellingPrice": 85.00,
  "reorderLevel": 100,
  "reorderQuantity": 500,
  "isActive": true,
  "trackInventory": true,
  "taxRate": 15.00
}
```

Warehouses

Method	Endpoint	Description
GET	`/inventory/warehouses`	List warehouses
GET	`/inventory/warehouses/{id}`	Get warehouse
POST	`/inventory/warehouses`	Create warehouse
GET	`/inventory/warehouses/{id}/stock`	Warehouse stock

Stock Movements

Method	Endpoint	Description
GET	`/inventory/stock-movements`	List movements
POST	`/inventory/stock-movements/receive`	Receive stock
POST	`/inventory/stock-movements/issue`	Issue stock
POST	`/inventory/stock-movements/transfer`	Transfer stock
POST	`/inventory/stock-movements/adjust`	Adjust stock

Transfer Stock:

```
POST /api/v2/inventory/stock-movements/transfer

{
  "productId": "prod_123",
  "fromWarehouseId": "wh_001",
  "toWarehouseId": "wh_002",
  "quantity": 100,
  "reference": "TRF-001",
  "reason": "Stock rebalancing"
}
```

Stock Takes

Method	Endpoint	Description
GET	`/inventory/stock-takes`	List stock takes
POST	`/inventory/stock-takes`	Create stock take
PUT	`/inventory/stock-takes/{id}`	Update counts
POST	`/inventory/stock-takes/{id}/complete`	Complete stock take

Sales & CRM API

Customers

Method	Endpoint	Description
GET	`/sales/customers`	List customers
GET	`/sales/customers/{id}`	Get customer
POST	`/sales/customers`	Create customer
PUT	`/sales/customers/{id}`	Update customer
GET	`/sales/customers/{id}/transactions`	Transaction history
GET	`/sales/customers/{id}/statement`	Customer statement

Create Customer:

```
POST /api/v2/sales/customers

{
  "name": "ABC Trading (Pty) Ltd",
  "tradingName": "ABC Trading",
  "registrationNumber": "2020/123456/07",
  "vatNumber": "4123456789",
  "contactPerson": "John Smith",
  "email": "john@abctrading.co.za",
  "phone": "+27 11 123 4567",
  "billingAddress": {
    "street": "123 Main Road",
    "city": "Johannesburg",
    "province": "Gauteng",
    "postalCode": "2001",
    "country": "ZA"
  },
  "paymentTerms": 30,
  "creditLimit": 50000.00,
  "priceListId": "pl_001"
}
```

Quotations

Method	Endpoint	Description
GET	`/sales/quotes`	List quotations
POST	`/sales/quotes`	Create quotation
PUT	`/sales/quotes/{id}`	Update quotation
POST	`/sales/quotes/{id}/send`	Send to customer
POST	`/sales/quotes/{id}/convert`	Convert to order

Sales Orders

Method	Endpoint	Description
GET	`/sales/orders`	List orders
POST	`/sales/orders`	Create order
POST	`/sales/orders/{id}/fulfill`	Fulfill order
POST	`/sales/orders/{id}/invoice`	Generate invoice

Invoices

Method	Endpoint	Description
GET	`/sales/invoices`	List invoices
POST	`/sales/invoices`	Create invoice
POST	`/sales/invoices/{id}/send`	Send invoice
POST	`/sales/invoices/{id}/payments`	Record payment
POST	`/sales/invoices/{id}/credit-note`	Create credit note
GET	`/sales/invoices/{id}/pdf`	Download PDF

Create Invoice:

```
POST /api/v2/sales/invoices

{
  "customerId": "cust_123",
  "invoiceDate": "2026-01-03",
  "dueDate": "2026-02-02",
  "reference": "PO-12345",
  "lineItems": [
    {
      "productId": "prod_001",
      "description": "Standard Widget",
      "quantity": 10,
      "unitPrice": 85.00,
      "discount": 5,
      "taxRate": 15
    }
  ],
  "notes": "Thank you for your business"
}
```

HR & Payroll API

Employees

Method	Endpoint	Description
GET	`/hr/employees`	List employees
GET	`/hr/employees/{id}`	Get employee
POST	`/hr/employees`	Create employee
PUT	`/hr/employees/{id}`	Update employee
POST	`/hr/employees/{id}/terminate`	Terminate employee

Leave Management

Method	Endpoint	Description
GET	`/hr/leave-requests`	List leave requests
POST	`/hr/leave-requests`	Submit leave request
POST	`/hr/leave-requests/{id}/approve`	Approve leave
POST	`/hr/leave-requests/{id}/reject`	Reject leave
GET	`/hr/employees/{id}/leave-balance`	Leave balances

Payroll

Method	Endpoint	Description
GET	`/hr/payroll/payruns`	List pay runs
POST	`/hr/payroll/payruns`	Create pay run
GET	`/hr/payroll/payruns/{id}`	Get pay run
POST	`/hr/payroll/payruns/{id}/calculate`	Calculate pay run
POST	`/hr/payroll/payruns/{id}/approve`	Approve pay run
POST	`/hr/payroll/payruns/{id}/process`	Process payments
GET	`/hr/employees/{id}/payslips`	Employee payslips

Create Pay Run:

```
POST /api/v2/hr/payroll/payruns

{
  "name": "January 2026 Monthly",
  "payPeriodStart": "2026-01-01",
  "payPeriodEnd": "2026-01-31",
  "payDate": "2026-01-25",
  "payFrequency": "monthly",
  "employeeIds": ["emp_001", "emp_002", "emp_003"]
}
```

SARS Integration

Method	Endpoint	Description
GET	`/hr/sars/emp201`	EMP201 submissions
POST	`/hr/sars/emp201/generate`	Generate EMP201
POST	`/hr/sars/emp201/{id}/submit`	Submit to SARS
GET	`/hr/sars/emp501`	EMP501 (annual)
GET	`/hr/sars/irp5`	IRP5 certificates

Webhooks

Overview

Webhooks allow you to receive real-time notifications when events occur in SiyaBusa.

Managing Webhooks

Method	Endpoint	Description
GET	`/webhooks`	List webhooks
POST	`/webhooks`	Create webhook
PUT	`/webhooks/{id}`	Update webhook
DELETE	`/webhooks/{id}`	Delete webhook
GET	`/webhooks/{id}/deliveries`	Delivery history

Create Webhook:

```
POST /api/v2/webhooks

{
  "url": "https://your-app.com/webhooks/siyabusa",
  "events": [
    "invoice.created",
    "invoice.paid",
    "payment.received",
    "customer.created"
  ],
  "secret": "your_webhook_secret",
  "isActive": true
}
```

Available Events

Category	Events
Invoices	`invoice.created`, `invoice.updated`, `invoice.paid`, `invoice.overdue`
Payments	`payment.received`, `payment.failed`
Customers	`customer.created`, `customer.updated`
Orders	`order.created`, `order.fulfilled`, `order.cancelled`
Inventory	`stock.low`, `stock.out`, `stock.received`
Payroll	`payrun.calculated`, `payrun.processed`

Webhook Payload

```
{
  "id": "evt_abc123",
  "type": "invoice.paid",
  "timestamp": "2026-01-03T10:30:00Z",
  "tenantId": "tenant_abc123",
  "data": {
    "invoiceId": "inv_xyz789",
    "invoiceNumber": "INV-001234",
    "customerId": "cust_123",
    "amount": 8500.00,
    "paymentDate": "2026-01-03"
  }
}
```

Verifying Webhooks

Verify webhook signatures using HMAC-SHA256:

```
const crypto = require('crypto');

function verifyWebhook(payload, signature, secret) {
  const hmac = crypto.createHmac('sha256', secret);
  const digest = hmac.update(payload).digest('hex');
  return crypto.timingSafeEqual(
    Buffer.from(signature),
    Buffer.from(digest)
  );
}
```

SDKs & Libraries

Official SDKs

Language	Package	Install
JavaScript/Node.js	`@siyabusa/sdk`	`npm install @siyabusa/sdk`
Python	`siyabusa`	`pip install siyabusa`
PHP	`siyabusa/sdk`	`composer require siyabusa/sdk`
C# / .NET	`SiyaBusa.SDK`	`dotnet add package SiyaBusa.SDK`

JavaScript SDK Example

```
const SiyaBusa = require('@siyabusa/sdk');

const client = new SiyaBusa({
  clientId: 'your_client_id',
  clientSecret: 'your_client_secret',
  tenantId: 'your_tenant_id',
  environment: 'production'
});

// List customers
const customers = await client.sales.customers.list({
  page: 1,
  limit: 20
});

// Create invoice
const invoice = await client.sales.invoices.create({
  customerId: 'cust_123',
  lineItems: [
    { productId: 'prod_001', quantity: 10, unitPrice: 85.00 }
  ]
});
```

Python SDK Example

```
from siyabusa import SiyaBusaClient

client = SiyaBusaClient(
    client_id='your_client_id',
    client_secret='your_client_secret',
    tenant_id='your_tenant_id'
)

# List customers
customers = client.sales.customers.list(page=1, limit=20)

# Create invoice
invoice = client.sales.invoices.create(
    customer_id='cust_123',
    line_items=[
        {'product_id': 'prod_001', 'quantity': 10, 'unit_price': 85.00}
    ]
)
```

Code Examples

cURL Examples

Authenticate:

```
curl -X POST https://api.siyabusa.com/api/v2/auth/token \
  -H "Content-Type: application/json" \
  -d '{
    "grant_type": "client_credentials",
    "client_id": "your_client_id",
    "client_secret": "your_client_secret",
    "tenant_id": "your_tenant_id"
  }'
```

List Invoices:

```
curl -X GET "https://api.siyabusa.com/api/v2/sales/invoices?status=unpaid" \
  -H "Authorization: Bearer {token}" \
  -H "X-Tenant-ID: your_tenant_id"
```

Create Customer:

```
curl -X POST https://api.siyabusa.com/api/v2/sales/customers \
  -H "Authorization: Bearer {token}" \
  -H "Content-Type: application/json" \
  -H "X-Tenant-ID: your_tenant_id" \
  -d '{
    "name": "New Customer Ltd",
    "email": "contact@newcustomer.co.za",
    "phone": "+27 11 123 4567"
  }'
```

Integration Patterns

Syncing Customers from External CRM

```
async function syncCustomers(externalCustomers) {
  for (const ext of externalCustomers) {
    // Check if customer exists
    const existing = await client.sales.customers.findByEmail(ext.email);

    if (existing) {
      // Update existing customer
      await client.sales.customers.update(existing.id, {
        name: ext.name,
        phone: ext.phone
      });
    } else {
      // Create new customer
      await client.sales.customers.create({
        name: ext.name,
        email: ext.email,
        phone: ext.phone,
        externalId: ext.id
      });
    }
  }
}
```

Processing Webhook Events

```
app.post('/webhooks/siyabusa', (req, res) => {
  // Verify signature
  const signature = req.headers['x-siyabusa-signature'];
  if (!verifyWebhook(req.rawBody, signature, WEBHOOK_SECRET)) {
    return res.status(401).send('Invalid signature');
  }

  const event = req.body;

  switch (event.type) {
    case 'invoice.paid':
      handleInvoicePaid(event.data);
      break;
    case 'stock.low':
      handleLowStock(event.data);
      break;
    default:
      console.log('Unhandled event:', event.type);
  }

  res.status(200).send('OK');
});
```

Changelog

Version 2.0 (January 2026)

- Initial v2 API release
- OAuth 2.0 authentication
- Multi-tenant support
- Comprehensive endpoint coverage
- Webhook support
- Rate limiting

Upcoming

- GraphQL endpoint (Q2 2026)
- Bulk operations API
- Real-time subscriptions

Support

API Support Channels

Channel	Purpose	Response Time
Documentation	docs.siyabusa.com	Self-service
Developer Forum	community.siyabusa.com	Community
Email Support	api-support@siyabusa.com	24 hours
Priority Support	Enterprise customers	4 hours

Status Page

Check API status at: **status.siyabusa.com**

Document Control

Version	Date	Author	Changes
1.0	January 2026	Engineering	Initial release

SiyaBusa ERP - Powering African Business